

Student: Jakub Kwaśniewski

Numer albumu 32430

Sprawozdanie: Dziel i zwyciężaj

Metody Programowania

Uczelnia: Politechnika Morska w Szczecinie

Data: 07.04.2026

Wstęp do zadania:

Celem ćwiczenia było zapoznanie się z metodą „dziel i zwyciężaj” oraz jej praktycznym zastosowaniem w programowaniu. W ramach zadania zaimplementowano algorytmy QuickSort oraz wyszukiwanie binarne, które pozwalają efektywnie przetwarzać dane.

Program umożliwia utworzenie dynamicznej tablicy liczb losowych, jej posortowanie oraz wyszukanie wybranego elementu. Dzięki temu możliwe było przeanalizowanie działania rekurencji oraz efektywności zastosowanych algorytmów.

Zadanie 1: Dynamiczne generowanie tablicy

1. Pozwoli użytkownikowi podać długość tablicy.

```
int n;  
cout << "Podaj rozmiar tablicy: ";  
cin >> n;
```

```
Podaj rozmiar tablicy: 5
```

Program pyta użytkownika o liczbę elementów tablicy. Dzięki temu możliwe jest dostosowanie rozmiaru tablicy dla potrzeb użytkownika.

2. Wygeneruje tablicę losowych liczb w podanym przez użytkownika zakresie.

```

int minVal, maxVal;
cout << "Podaj zakres min: ";
cin >> minVal;
cout << "Podaj zakres max: ";
cin >> maxVal;

int* arr = new int[n];
cout << "Tablica: ";
for (int i = 0; i < n; i++) {
    arr[i] = rand() % (maxVal - minVal + 1) + minVal;
    cout << arr[i] << " ";
}

```

```

Podaj rozmiar tablicy: 8
Podaj zakres min: 1
Podaj zakres max: 10
Tablica: 2 8 5 1 10 5 9 9

```

Tablica jest wypełniana losowymi wartościami z podanego zakresu min i max. Użyto funkcji rand() a każda liczba jest niezależnie losowana.

3. Posortuje tablicę.

```

quickSort(arr, 0, n - 1);
cout << "Posortowana tablica: ";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
cout << endl;

```

```

Podaj rozmiar tablicy: 10
Podaj zakres min: 1
Podaj zakres max: 12
Tablica: 6 12 11 5 6 5 7 7 11 9
Posortowana tablica: 5 5 6 6 7 7 9 11 11 12

```

Tablica jest uporządkowana przy użyciu QuickSort. Algorytm dzieli tablicę na mniejsze części, sortuje je i łączy w całość.

4. Pozwoli użytkownikowi wpisać liczbę do wyszukania i zwróci jej indeks lub informację o jej braku.

```
int target;  
cout << "Podaj liczbę do znalezienia: " << endl;  
cin >> target;  
int result = binarySearch(arr, 0, n - 1, target);  
  
if (result != -1)  
    cout << "Element znaleziony na indeksie " << result << endl;  
else  
    cout << "Element nie znaleziony" << endl;  
return 0;
```

```
Podaj rozmiar tablicy: 10  
Podaj zakres min: 1  
Podaj zakres max: 12  
Tablica: 6 12 11 5 6 5 7 7 11 9  
Posortowana tablica: 5 5 6 6 7 7 9 11 11 12  
Podaj liczbę do znalezienia:  
6  
Element znaleziony na indeksie 2
```

Użytkownik podaje liczbę a program szuka jej w tablicy przy pomocy wyszukiwania binarnego i pokazuje indeks lub informuje, że jej nie ma.

Zadanie 2: Analiza czasowa

Napisz program, który porówna czas działania QuickSort oraz standardowej funkcji sort() z biblioteki STL dla dużych zestawów danych (np. 1 000 000 liczb losowych). Wykonaj co najmniej 3 pomiary. Umieść je w tabeli. Zapisz wnioski.

```
const int N = 2000000;
int* arr1 = new int[N];

for (int i = 0; i < N; i++) arr1[i] = rand();
auto start = high_resolution_clock::now();
quickSort(arr1, 0, N - 1);
auto stop = high_resolution_clock::now();
chrono::duration<double> duration = stop - start;
cout << "Czas QuickSort: " << duration.count() << " sekund" << endl;

for (int i = 0; i < N; i++) arr1[i] = rand();
start = high_resolution_clock::now();
sort(arr1, arr1 + N);
stop = high_resolution_clock::now();
duration = stop - start;
cout << "Czas sort: " << duration.count() << " sekund" << endl;
/*
czas losowania dla QuickSort oraz dla dla funkcji sort()
QuickSort:
1 losowanie dla 1000000 liczb losowanych - 0.170881 sekundy
2 losowanie dla 1500000 liczb losowanych - 0.272088 sekundy
3 losowanie dla 2000000 liczb losowanych - 0.416912 sekundy

sort():
1 losowanie dla 1000000 liczb losowanych - 0.328570 sekundy
2 losowanie dla 1500000 liczb losowanych - 0.530816 sekundy
3 losowanie dla 2000000 liczb losowanych - 0.638574 sekundy
*/
return 0;
```

```
Czas QuickSort: 0.433494 sekund
Czas sort: 0.642332 sekund
```

Czas działania rośnie wraz ze wzrostem liczby elementów QuickSort i sort() dobrze sortują duże tablice a różnice w czasie zależą od implementacji i wielkości danych.

a) posortowane;

```

int N = 1000000;
int* arr = new int[N];

for (int i = 0; i < N; i++) arr[i] = i;

auto start = high_resolution_clock::now();
quickSort(arr, 0, N - 1);
auto stop = high_resolution_clock::now();
chrono::duration<double> duration = stop - start;
cout << "Czas QuickSort dla posortowanej tablicy: " << duration.count() << " sekund" << endl;

for (int i = 0; i < N; i++) arr[i] = i;

start = high_resolution_clock::now();
sort(arr, arr + N);
stop = high_resolution_clock::now();
duration = stop - start;
cout << "Czas sort dla posortowanej tablicy: " << duration.count() << " sekund" << endl;
/*
czas losowania dla QuickSort oraz dla dla funkcji sort()
QuickSort:
1 losowanie dla 1400 liczb losowanych - 0.005349 sekundy
2 losowanie dla 1600 liczb losowanych - 0.0067998 sekundy
3 losowanie dla 2000000 liczb losowanych - 0.008701 sekundy

sort():
1 losowanie dla 1000000 liczb losowanych - 0.0001428 sekundy
2 losowanie dla 1500000 liczb losowanych - 0.0001737 sekundy
3 losowanie dla 2000000 liczb losowanych - 0.0001796 sekundy
*/
return 0;

```

Czas QuickSort dla posortowanej tablicy: 0.0053552 sekund
Czas sort dla posortowanej tablicy: 0.0001421 sekund

Czas działania rośnie wraz ze wzrostem liczby elementów QuickSort jest wolniejszy od sort() na małych i dużych tablicach a sort() działa bardzo szybko nawet dla milionów elementów.

b) posortowane w przeciwnym kierunku.

```

int N = 1800;
int* arr = new int[N];

for (int i = 0; i < N; i++) arr[i] = N - i;

auto start = high_resolution_clock::now();
quickSort(arr, 0, N - 1);
auto stop = high_resolution_clock::now();
chrono::duration<double> duration = stop - start;
cout << "Czas QuickSort malejaco: " << duration.count() << " sekund" << endl;

for (int i = 0; i < N; i++) arr[i] = N - i;

start = high_resolution_clock::now();
sort(arr, arr + N);
stop = high_resolution_clock::now();
duration = stop - start;
cout << "Czas sort malejaco: " << duration.count() << " sekund" << endl;
/*
    czas losowania dla QuickSort oraz dla dla funkcji sort()
    QuickSort:
    1 losowanie dla 1400 liczb losowanych - 0.0034267 sekundy
    2 losowanie dla 1600 liczb losowanych - 0.0048778 sekundy
    3 losowanie dla 1800 liczb losowanych - 0.0056755 sekundy

    sort():
    1 losowanie dla 1400 liczb losowanych - 0.0001668 sekundy
    2 losowanie dla 1600 liczb losowanych - 0.0002035 sekundy
    3 losowanie dla 1800 liczb losowanych - 0.0002302 sekundy
*/
return 0;

```

Czas QuickSort malejaco: 0.0056755 sekund
Czas sort malejaco: 0.0002302 sekund

QuickSort działa wolniej na tablicach posortowanych malejaco natomiast sort()
szybko poradzi sobie niezależnie od kierunku sortowania.

Tabele pomiarów do zadanie 2:

Zakres liczb losowanych	Czas działania QuickSort w sekundach[s]	Czas działania Sort w sekundach[s]
1000000	0,170881[s]	0,328570[s]
1500000	0,272088[s]	0,530816[s]
2000000	0,416912[s]	0,638574[s]

Zakres liczb losowanych posortowanych	Czas działania QuickSort w sekundach[s]	Czas działania Sort w sekundach[s]
1400	0,005349[s]	0,0001428[s]
1600	0,0067998[s]	0,0001737[s]
1800	0,0087010[s]	0,0001796[s]

liczby posortowane w przeciwnym kierunku	Czas działania QuickSort w sekundach[s]	Czas działania Sort w sekundach[s]
1400	0,0034267[s]	0,0001668[s]
1600	0,0048778[s]	0,0002035[s]
1800	0,0056755[s]	0,0002302[s]

Zadanie 3: Analiza liczby operacji

```
long long swapCount = 0;
long long binarySearchCalls = 0;

int binarySearch(int arr[], int left, int right, int target) {
    binarySearchCalls++;
    if (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target)
            return mid;
        else if (arr[mid] > target)
            return binarySearch(arr, left, mid - 1, target);
        else
            return binarySearch(arr, mid + 1, right, target);
    }
    return -1;
}

void swap(int& a, int& b) {
    swapCount++;
    int temp = a;
    a = b;
    b = temp;
}
```

```
int sizes[] = { 10, 100, 1000, 10000, 100000, 1000000 };

for (int sz : sizes) {
    int* arr = new int[sz];

    for (int i = 0; i < sz; i++) arr[i] = rand();

    swapCount = 0;
    auto startQS = high_resolution_clock::now();
    quickSort(arr, 0, sz - 1);
    auto stopQS = high_resolution_clock::now();
    double timeQS = duration<double>(stopQS - startQS).count();

    binarySearchCalls = 0;
    int target = arr[sz / 2];
    auto startBS = high_resolution_clock::now();
    binarySearch(arr, 0, sz - 1, target);
    auto stopBS = high_resolution_clock::now();
    double timeBS = duration<double>(stopBS - startBS).count();

    double totalTime = timeQS + timeBS;

    cout << "Rozmiar tablicy: " << sz << endl; ;
    cout << "Czas QuickSort: " << timeQS << " s " << endl;
    cout << "Liczba operacji swap: " << swapCount << " " << endl;
    cout << "Czas BinarySearch: " << timeBS << " s " << endl;
    cout << "Liczba wywołań BinarySearch: " << binarySearchCalls << " " << endl;
    cout << "Czas łączny: " << totalTime << " s " << endl;
    cout << endl;
}

return 0;
}
```

```
Rozmiar tablicy: 10
Czas QuickSort: 5.2e-06 s
Liczba operacji swap: 17
Czas BinarySearch: 1e-07 s
Liczba wywołań BinarySearch: 3
Czas łączny: 5.3e-06 s

Rozmiar tablicy: 100
Czas QuickSort: 6.6e-06 s
Liczba operacji swap: 365
Czas BinarySearch: 2e-07 s
Liczba wywołań BinarySearch: 6
Czas łączny: 6.8e-06 s

Rozmiar tablicy: 1000
Czas QuickSort: 9.1e-05 s
Liczba operacji swap: 5725
Czas BinarySearch: 2e-07 s
Liczba wywołań BinarySearch: 9
Czas łączny: 9.12e-05 s

Rozmiar tablicy: 10000
Czas QuickSort: 0.0011977 s
Liczba operacji swap: 83858
Czas BinarySearch: 2e-07 s
Liczba wywołań BinarySearch: 12
Czas łączny: 0.0011979 s

Rozmiar tablicy: 100000
Czas QuickSort: 0.0143548 s
Liczba operacji swap: 1042991
Czas BinarySearch: 1e-07 s
Liczba wywołań BinarySearch: 1
Czas łączny: 0.0143549 s

Rozmiar tablicy: 1000000
Czas QuickSort: 0.177597 s
Liczba operacji swap: 10856579
Czas BinarySearch: 2e-07 s
Liczba wywołań BinarySearch: 1
Czas łączny: 0.177598 s
```

Czas QuickSort rośnie z rozmiarem tablicy i liczbą swapów, a BinarySearch pozostaje bardzo szybki i powoli rośnie.

Tabela do zadania 3:

Czas QuickSort[s]	Liczba Operacji swap	Czas BinarySearch[s]	Liczba wywołań BinarySearch	Czas łączny[s]
4.5e-06[s]	17	2e-07[s]	3	4.7e-06[s]
6.6e-06[s]	365	2e-07[s]	6	6.8e-06[s]
9.07e-05[s]	5725	1e-07[s]	9	9.08e-05[s]
0.0011599[s]	83858	1e-07[s]	12	0.00116[s]
0.0141752[s]	1042991	1e-07[s]	1	0.0141753[s]
0.184671[s]	10856579	1e-07[s]	1	0.184671[s]

Wnioski do zadania:

Ćwiczenie pokazało, że technika dziel i zwyciężaj jest niezwykle pomocna przy pracy z dużymi zbiorami danych, bo pozwala znacząco przyspieszyć działanie programu. Podczas testów zauważyłem, że QuickSort radzi sobie szybko z losowymi liczbami, ale traci swoją przewagę przy danych już poukładanych wtedy bezpieczniej jest użyć standardowej funkcji sort() która działa stabilnie w każdych warunkach. Z kolei wyszukiwanie binarne okazało się bezkonkurencyjne, ponieważ znajduje konkretną wartość w ogromnej tablicy niemal natychmiast wykonując tylko kilka prostych kroków.